

MANUAL DE DEPLOY

Manual de Instalación y Despliegue Local de CarFind

INSTITUCION: Escuela Tecnica N° 32 DE14 "Gral Jose de San Martin"

MATERIA/TALLER: Prácticas Profesionalizantes

DOCENTES: Gonzalo Nicolás Consorti, Damian Olaso y Luciano Alderete

ALUMNOS: Daiana Rivera, Ignacio Cordoba, Leandro Delgado, Mauricio Arzabe, Kiara Hernandez, Samuel Vera y Ludmila Rovner

CURSO: 6°1° Turno Mañana Computación

Índice

1. Introducción y contexto

1.1. Objetivo del manual

1.2. Alcance del procedimiento

1.3. Descripción general del sistema

1.4. Arquitectura del sistema

1.4.1. Entorno de desarrollo local

1.4.2. Infraestructura en la nube 1.5. Glosario de términos técnicos 2. Requisitos previos y configuración 2.1. Requisitos de hardware y software 2.2. Instalación y verificación

2.3. Estructura de archivos

2.4. Archivos de configuración y .env 3. Instalación y ejecución local 3.1. Configuración de la base de datos

3.1.1. Inicialización en XAMPP

3.1.2. Configuración de puerto

3.1.3. Importación de scripts

3.1.4. Verificación de datos

3.2. Configuración del backend

3.2.1. Conexión a base de datos

3.2.2. Ejecución Maven / Eclipse

3.2.3. Rutas y archivos multimedia

3.2.4. Verificación de la API

3.3. Configuración del frontend 3.3.1. Instalación de dependencias 3.3.2. URL del backend

3.3.3. Servidor Vite

3.3.4. Modos API Real y Mock

4. Validación e integración

4.1. Cuentas y roles de prueba

4.2. Comunicación frontend-backend 4.3. Endpoints y respuestas de la API 4.4. Operaciones en base de datos 4.5. Pruebas funcionales de integración

5. Despliegue en infraestructura Cloud 5.1. Base de datos gestionada (Aiven) 5.2. Despliegue del backend (Render) 5.3. Despliegue del frontend (Vercel) 5.4. Variables de entorno en la nube 5.5. Verificación de aplicación

6. Anexos y documentación técnica

6.1. Comandos de instalación

6.2. Archivos de configuración

6.3. Documentación API (Swagger)

6.4. Referencia API_CARFIND

1. Introducción y contexto

1.1. Objetivo del manual

El presente documento tiene como objetivo principal guiar la instalación, configuración, ejecución y despliegue del sistema CarFind, estructurando los procedimientos tanto para el entorno de desarrollo local como para su infraestructura en la nube. Asimismo, funciona como una guía técnica unificada para desarrolladores, administradores y coordinadores del proyecto, garantizando la correcta integración entre la base de datos, el backend y el forntend.

1.2. Alcance del procedimiento

El alcance de este procedimiento abarca los siguientes componentes y fases del sistema:

Configuración e instalación en entorno local:

Inicialización y carga de datos en la base de datos relacional MySQL/MariaDB (compraventa en el puerto 3307), incluyendo la ejecución del script de estructura compraventa.sql y el script automático de inicialización integracion.sql.

Ejecución del servidor backend en Java / Spring Boot (puerto 8080) desde Eclipse o mediante consola Maven (mvn spring-boot:run), junto con el ruteo de APIs en /api/web/** y el servidor de archivos multimedia en /uploads/**

Configuración y arranque del servidor frontend en React + Vite (puerto 5173), manejando las variables de entorno (.env) para alternar entre datos simulados de prueba (VITE_USE_MOCK=true) y conexión a la API real (VITE_USE_MOCK=false).

Verificación funcional mediante cuentas de prueba (Usuario y Administrador) y mapeo entre pantallas, endpoints /api/web/** y tablas de la base de datos.

Infraestructura y despliegue en la nube:

Integración de servicios distribuidos Cloud: Aiven para la base de datos, Render para el backend y Vercel para el frontend.

Documentación técnica de API:

Referencia de endpoints y operaciones CRUD documentadas mediante la especificación Swagger.
Límites y exclusiones del procedimiento (MVP):

El sistema está enfocado exclusivamente en la comercialización de vehículos completos; no contempla la publicación ni venta de repuestos o autopartes sueltas.

No incluye procesamiento ni cobro directo a través de pasarelas de pago integradas en esta versión; los pagos, transferencias y trámites de transferencia se acuerdan fuera de la plataforma. No realiza validaciones automatizadas externas en tiempo real con bases de datos gubernamentales; la información técnica, legal y de VTV corresponde a datos declarados por el usuario.

1.3. Descripción general del sistema

CarFind es una plataforma web especializada diseñada para la gestión eficiente de la compra y venta de vehículos nuevos y usados. El sistema fue concebido para resolver las limitaciones detectadas en plataformas generalistas (como Facebook Marketplace y Mercado Libre), las cuales carecen de estructuras y filtros específicos para el rubro automotor, lo que genera publicaciones poco estructuradas o confusas.

CarFind centraliza la experiencia a través de una arquitectura orientada al usuario que abarca:
Autenticación y perfiles: Registro de usuarios, inicio de sesión seguro y gestión de perfiles con diferenciación de roles (Comprador, Vendedor y Administrador).

Buscador y catálogo: Motor de búsqueda con filtros avanzados por marca, modelo, precio, año, kilometraje y características técnicas.

Ficha técnica y publicaciones: Creación de avisos con previsualización, carga de fotografías y detalle completo del vehículo (motor, transmisión, combustible, documentación declarada). Interacción y moderación: Gestión de favoritos/wishlist, mensajería interna o acceso a WhatsApp para contactar vendedores, e interfaz de administración para moderar publicaciones e informes.

1.4. Arquitectura del sistema

1.4.1. Entorno de desarrollo local

El entorno local implementa una arquitectura cliente-servidor estructurada en tres capas conectadas de la siguiente manera:

Navegador → Frontend (React + Vite, :5173) &xrightarrow{/api/** y /uploads/**} Backend (Spring Boot, :8080) → Base de Datos MySQL/MariaDB (:3307)

Base de datos (MySQL / MariaDB): Ejecutada sobre XAMPP en el puerto 3307. Requiere la creación de la base compraventa e importación del script Base_De_Datos_CarFind/compraventa.sql. Al iniciar el backend, este ejecuta automáticamente integracion.sql para incorporar datos obligatorios para el frontend (roles ADMIN, foto de perfil, tipos de reporte, etc.).

Backend (Spring Boot / Java): Ubicado en Backend_CarFind/carfind, se ejecuta en el puerto 8080 desde Eclipse (CarfindApplication.java) o por consola Maven (mvn spring-boot:run). Expone la API consumida por el frontend bajo /api/web/** (paquete com.back.carfind.web), mantiene los CRUD originales en /api/** y sirve las imágenes guardadas en uploads/ mediante la ruta /uploads/**.

Frontend (React + Vite): Ubicado en Frontend_CarFind, se ejecuta mediante npm install y npm run dev en el puerto 5173. Su archivo .env establece VITE_USE_MOCK=false y VITE_API_URL=/api/web para comunicarse directamente con el servidor backend.

1.4.2. Infraestructura de despliegue en la nube

Para el entorno de producción o staging en la nube, la arquitectura se distribuye en tres plataformas dedicadas:

Aiven: Servicio gestionado para el alojamiento y administración de la Base de Datos relacional. Render: Servidor Cloud PaaS para la compilación, ejecución y despliegue del Backend en Spring Boot. Vercel: Plataforma de hosting y despliegue continuo para la aplicación Frontend en React + Vite.

1.5. Glosario de términos técnicos

React: Biblioteca de JavaScript utilizada para la construcción de interfaces de usuario dinámicas e interactivas en el frontend.

Vite: Herramienta de desarrollo y empaquetador rápido para aplicaciones web frontend que gestiona el servidor local en el puerto 5173.

Spring Boot: Framework basado en Java utilizado para desarrollar el servidor backend y crear APIs REST escalables.

MySQL / MariaDB: Sistemas de gestión de bases de datos relacionales utilizados para el almacenamiento estructurado de la información.

XAMPP: Paquete de software independiente que incluye Apache, MariaDB/MySQL y phpMyAdmin para entornos de desarrollo local.

Maven: Herramienta de gestión de dependencias y construcción de proyectos Java utilizada para compilar y ejecutar el backend (mvn spring-boot:run).

API REST: Interfaz de programación de aplicaciones que permite el intercambio de datos entre cliente y servidor mediante solicitudes HTTP y formato JSON.

Proxy de Vite: Redirección configurada en el frontend para canalizar las peticiones /api/** y /uploads/** desde el puerto 5173 hacia el backend en el puerto 8080.

Mock Data: Datos simulados integrados en el frontend (VITE_USE_MOCK=true) que permiten probar la interfaz sin necesidad de tener el backend en ejecución.

Swagger: Especificación y conjunto de herramientas para la documentación interactiva y pruebas de endpoints de APIs REST.

Aiven: Servicio en la nube gestionado para el alojamiento y administración de bases de datos.
Render: Plataforma Cloud PaaS para alojar y ejecutar servicios web y aplicaciones backend. Vercel: Plataforma de despliegue en la nube optimizada para aplicaciones frontend y sitios web.

2. Requisitos previos y configuración del entorno

2.1. Requisitos de hardware y software

Requisitos de Hardware

Para garantizar la ejecución fluida y en simultáneo de la base de datos (MySQL/MariaDB en XAMPP), el servidor backend (Spring Boot / Java), el servidor frontend (React + Vite), el IDE de desarrollo y el navegador web, se requieren equipos (laptops o PCs de escritorio) de 2020/2021 en adelante (con perfil de desarrollo o gaming):

- Procesador (CPU): Intel Core i5 (10.ª generación o superior) / AMD Ryzen 5 (serie 3000/4000 o superior) con al menos 4 núcleos y 8 hilos.
- Memoria RAM: 16 GB DDR4/DDR5 (mínimo recomendado para mantener la JVM de Java, los servicios de Node.js y la base de datos en ejecución simultánea).
- Almacenamiento: Disco de estado sólido (SSD SATA III o NVMe) con al menos 20 GB de espacio libre disponible para dependencias (node_modules, repositorio Maven) y archivos multimedia (uploads/).
- Conectividad de Red: Conexión a internet estable de alta velocidad mediante placa Wi-Fi 5 / Wi-Fi 6 o puerto Ethernet Gigabit (1000 Mbps).

Requisitos de Software

- Servidor de Base de Datos: XAMPP para la ejecución de MySQL / MariaDB y gestión mediante phpMyAdmin.
- Entorno Backend: Java JDK y Apache Maven (o un entorno de desarrollo integrado como Eclipse habilitado para aplicaciones Spring Boot).
- Entorno Frontend: Node.js junto con el gestor de paquetes npm.
- Navegador Web: Navegador moderno (Chrome, Edge, Firefox) para el acceso y prueba de la interfaz web.

2.2. Instalación y verificación de herramientas

Para verificar que las herramientas necesarias estén correctamente instaladas en el sistema, se deben ejecutar los siguientes pasos de comprobación:

1. Node.js y npm (Frontend):

    Comprobar la instalación ejecutando los comandos `node -v` y `npm -v` desde la consola de comandos.

    Versión requerida: Node.js v18.x LTS (o superior) y npm v9.x (necesarias para el soporte de Vite y React).

2. Java JDK y Apache Maven (Backend):

    Verificar la disponibilidad del compilador de Java con `java -version` y la herramienta de construcción mediante `mvn -v`.

    Versión requerida: Java JDK 17 LTS (o JDK 21 LTS) y Apache Maven 3.8.x (estándar compatible con Spring Boot 3.x).

3. XAMPP / MariaDB / MySQL (Base de datos):

    Instalar el paquete XAMPP e iniciar el servicio de MySQL/MariaDB asegurando su configuración en el puerto 3307.

    Validar el acceso al panel administrativo ingresando a phpMyAdmin a través del navegador. 4. IDE o Consola de ejecución:

    Configurar Eclipse (mediante Run As Java Application / Spring Boot App) o habilitar la terminal de comandos para ejecución con Maven (`mvn spring-boot:run`).

2.3. Obtención y estructura de los archivos del proyecto

Obtención del código fuente: Para obtener los archivos del proyecto CarFind, clonar el repositorio oficial utilizando la herramienta Git desde la consola de comandos:

`git clone` <URL_DEL_REPOSITORIO>

cd CarFind

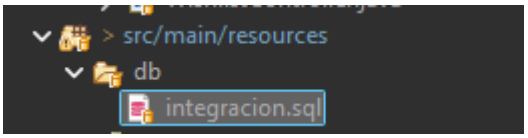
Estructura general de directorios: El proyecto se organiza en tres bloques principales que sostienen la arquitectura cliente-servidor:

Backend_CarFind/carfind: Contiene el desarrollo del servidor en Java / Spring Boot.

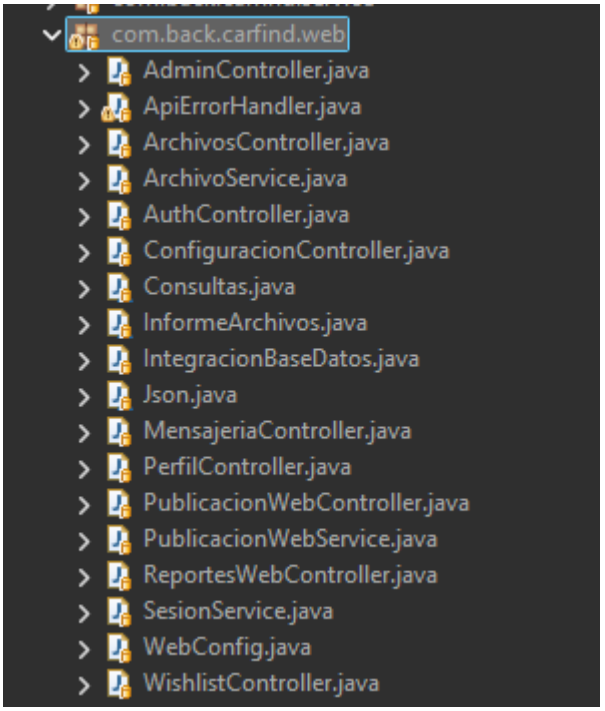
`src/main/resources/application.properties`: Archivo de configuración central de parámetros del servidor y base de datos.



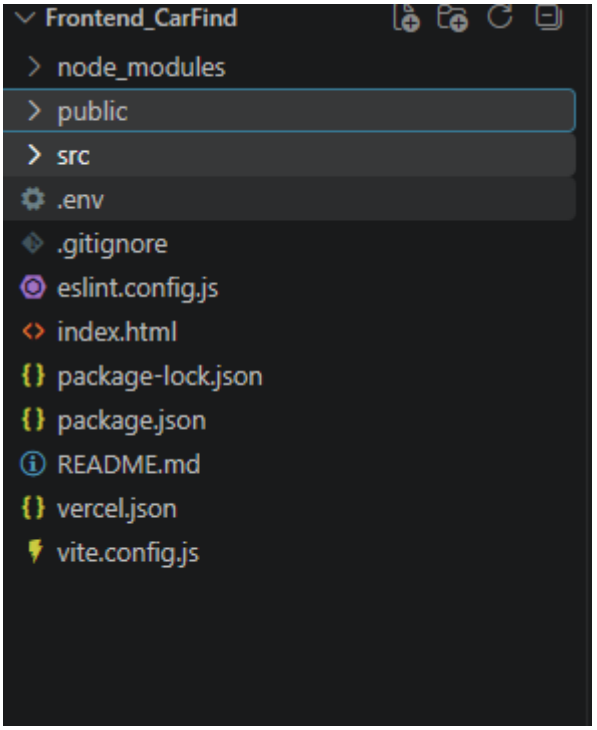
`src/main/resources/db/integracion.sql`: Script SQL automático ejecutado por el backend para incorporar datos iniciales obligatorios.



`src/main/java/com/back/carfind/web`: Paquete Java donde se ubican los controladores REST dedicados a la API consumida por la interfaz (`/api/web/**`).

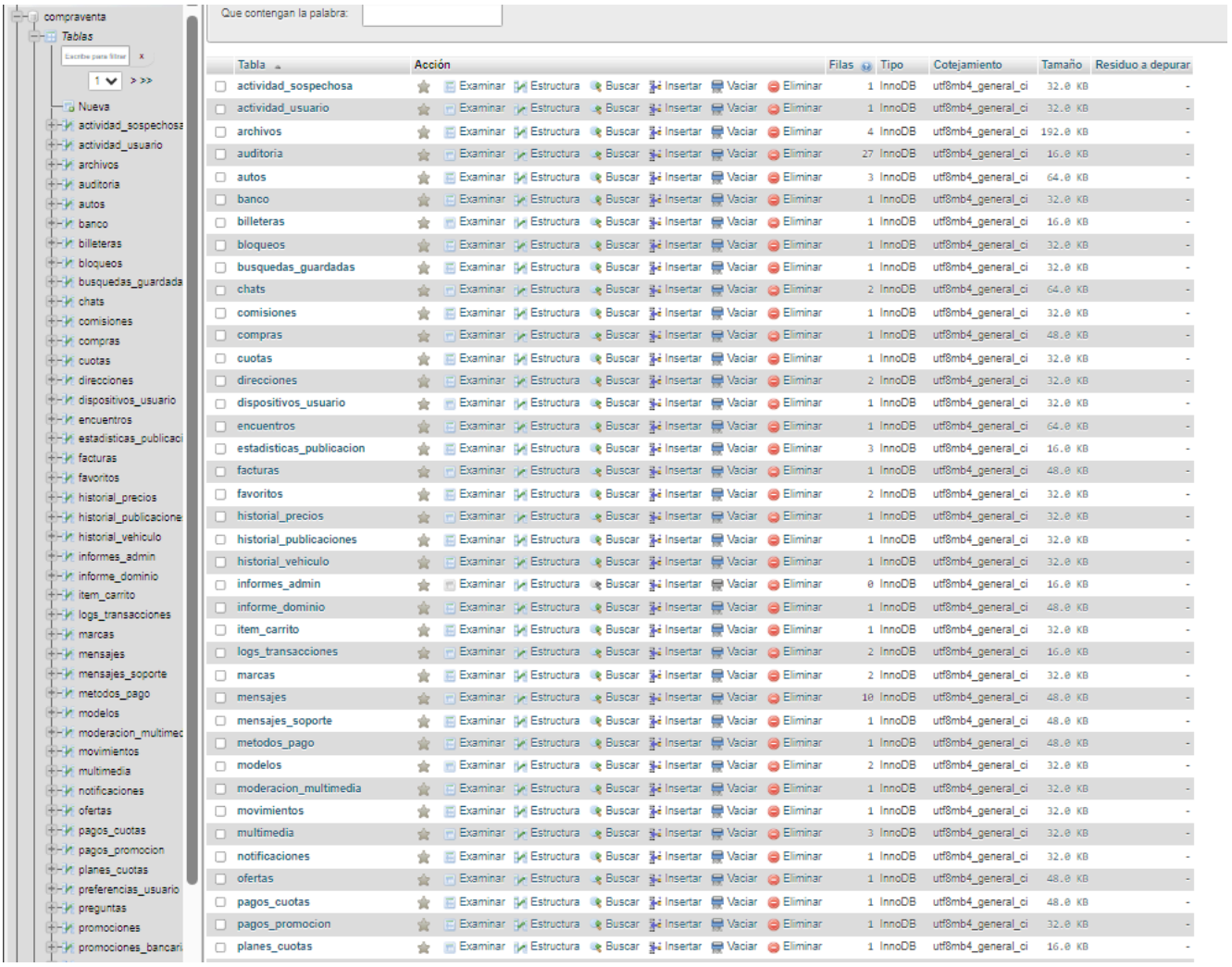


Frontend_CarFind: Contiene la aplicación de interfaz de usuario desarrollada con React y Vite. `.env`: Archivo de variables de entorno para controlar los modos de ejecución del cliente. `vite.config.js`: Archivo de configuración del servidor de desarrollo Vite que incluye la regla de proxy para el backend.



Base_De_Datos_CarFind: Alberga los archivos de base de datos.

`compraventa.sql`: Script SQL con la estructura de tablas y datos iniciales para la creación e importación de la base de datos compraventa.



2.4. Archivos de configuración y variables de entorno

1. Configuración del Backend (application.properties): Ubicado en

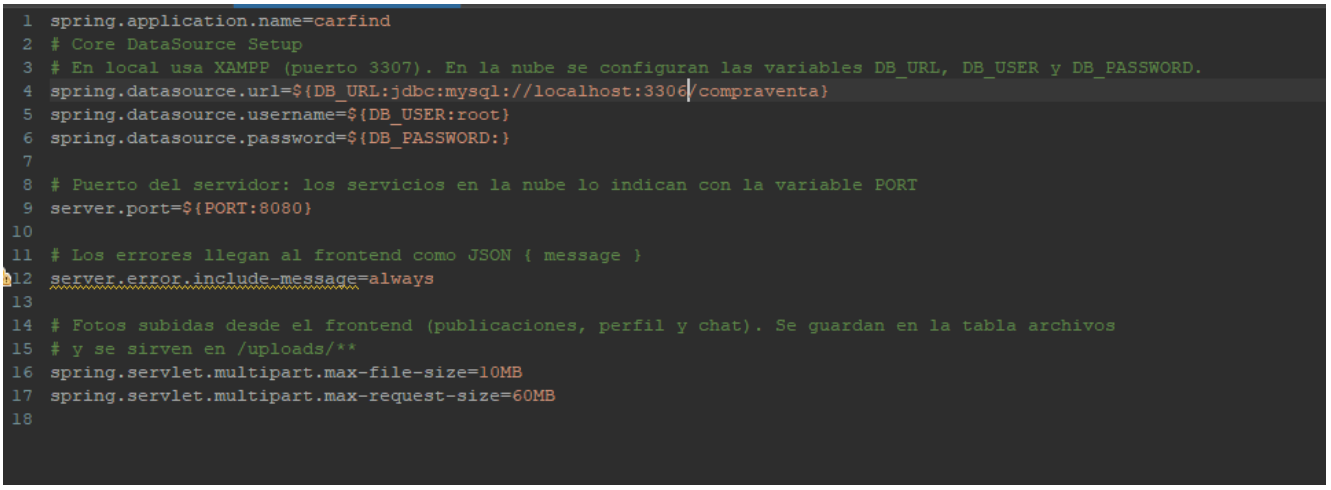
   `Backend_CarFind/carfind/src/main/resources/application.properties`, este archivo parametriza la conexión hacia la base de datos local:

   Puerto de base de datos: 3307.

   Credenciales de acceso: Usuario root sin contraseña.

   Nombre de la base de datos: compraventa.

   Carga automática de scripts: Al iniciar, el backend ejecuta de forma automática el archivo integracion.sql, el cual agrega elementos clave para el correcto funcionamiento del frontend (como foto de perfil por defecto, tipos de reporte, rol ADMIN, respuestas administrativas e informes). Este script está diseñado para ejecutarse múltiples veces sin generar conflictos.



```
spring.application.name=carfind
# Core DataSource Setup
# En local usa XAMPP (puerto 3307). En la nube se configuran las variables DB_URL, DB_USER y DB_PASSWORD.
spring.datasource.url=${DB_URL:jdbc:mysql://localhost:3306/compraventa}
spring.datasource.username=${DB_USER:root}
spring.datasource.password=${DB_PASSWORD:}

# Puerto del servidor: los servicios en la nube lo indican con la variable PORT
server.port=${PORT:8080}

# Los errores llegan al frontend como JSON { message }
server.error.include-message=always

# Fotos subidas desde el frontend (publicaciones, perfil y chat). Se guardan en la tabla archivos
# y se sirven en /uploads/**
spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=60MB

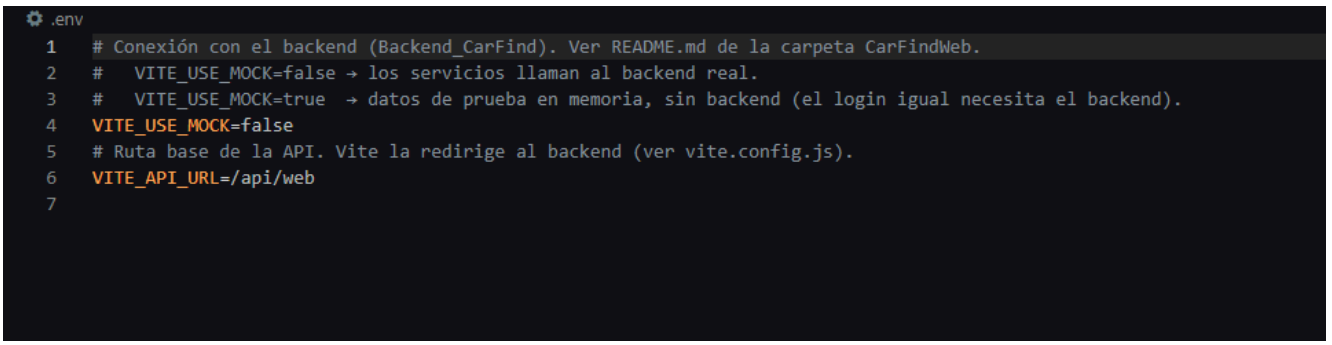
```

2. Variables de Entorno del Frontend (.env): Ubicado en `Frontend_CarFind/.env`, define cómo se comportará la aplicación en el navegador:

   `VITE_USE_MOCK=false`: Fuerza al frontend a conectarse directamente con el backend real para leer y persistir datos en la base de datos MySQL/MariaDB.

   `VITE_USE_MOCK=true`: Activa un conjunto de datos de prueba en memoria (mock data), lo que permite evaluar el comportamiento de las pantallas sin necesidad de tener el backend en ejecución.

`VITE_API_URL=/api/web`: Establece la ruta base para consumir la API alojada en el paquete com.back.carfind.web.



3. Configuración del Proxy de Vite (vite.config.js): Ubicado en `Frontend_CarFind/vite.config.js`, gestiona el ruteo de peticiones y previene problemas de origen cruzado (CORS):

   Redirección de peticiones: Redirige automáticamente las llamadas enviadas a `/api/**` y `/uploads/**` desde el puerto 5173 (Frontend) hacia el puerto 8080 (Backend Spring Boot). Despliegue alternativo: En caso de empaquetar el backend como un archivo WAR para su ejecución en un servidor Apache Tomcat con un context path distinto, es necesario ajustar la variable BACKEND_URL dentro de este archivo.

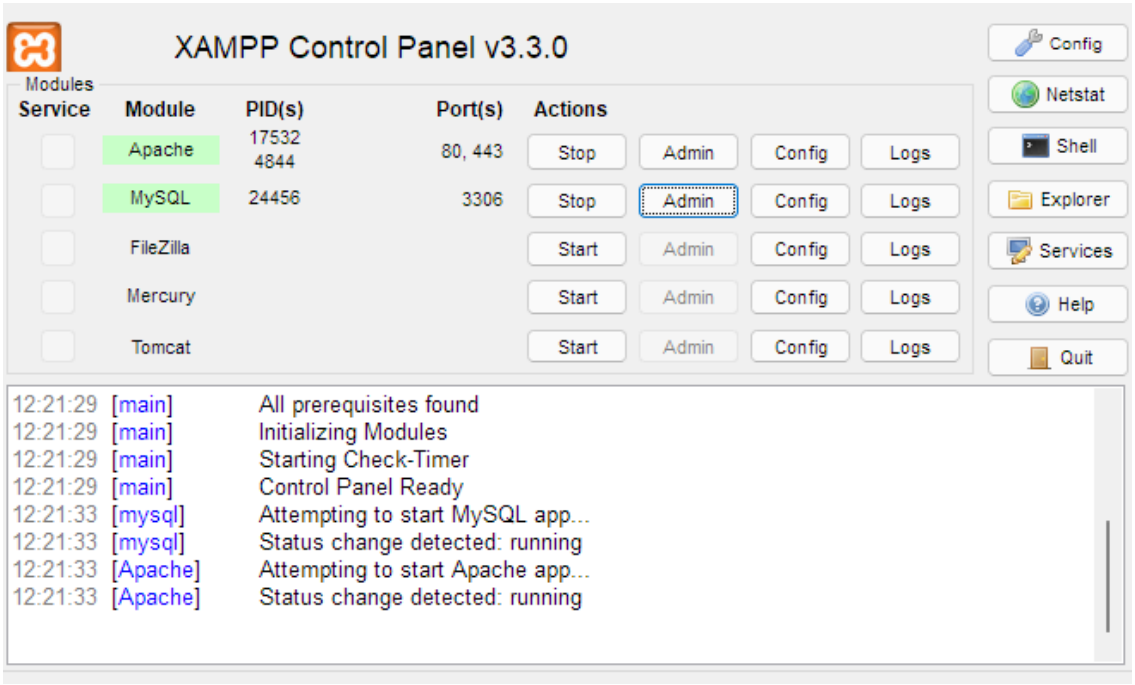


3. Instalación y ejecución en entorno local

3.1. Configuración de la base de datos

3.1.1. Inicialización de MySQL mediante XAMPP

Para poner en marcha la base de datos relacional del sistema CarFind, se debe iniciar el servicio de MySQL / MariaDB a través del panel de control de XAMPP.



3.1.2. Configuración del puerto de conexión

Puerto del servicio: El servicio debe ejecutarse en el puerto 3306.

Credenciales de acceso: Se accede con el usuario root sin contraseña.

El servidor local de base de datos MariaDB / MySQL en XAMPP se ejecuta en la dirección 127.0.0.1:3307. Si XAMPP tiene asignado por defecto el puerto 3306 (ocupado habitualmente por otra instalación local de MySQL), se debe reconfigurar el puerto al 3307 siguiendo estos pasos:

1. En el panel de control de XAMPP, hacer clic en el botón Config de la fila de MySQL y seleccionar el archivo my.ini.
2. Buscar las líneas donde figura port=3306 (tanto en la sección [client] como en [mysqld]) y reemplazar 3306 por 3307.
3. Guardar el archivo my.ini y reiniciar el servicio de MySQL desde el panel de XAMPP. 4.

Validar que la cadena de conexión en application.properties mantenga la URL `jdbc:mysql://localhost:3306/compraventa` para coincidir con la instancia configurada.

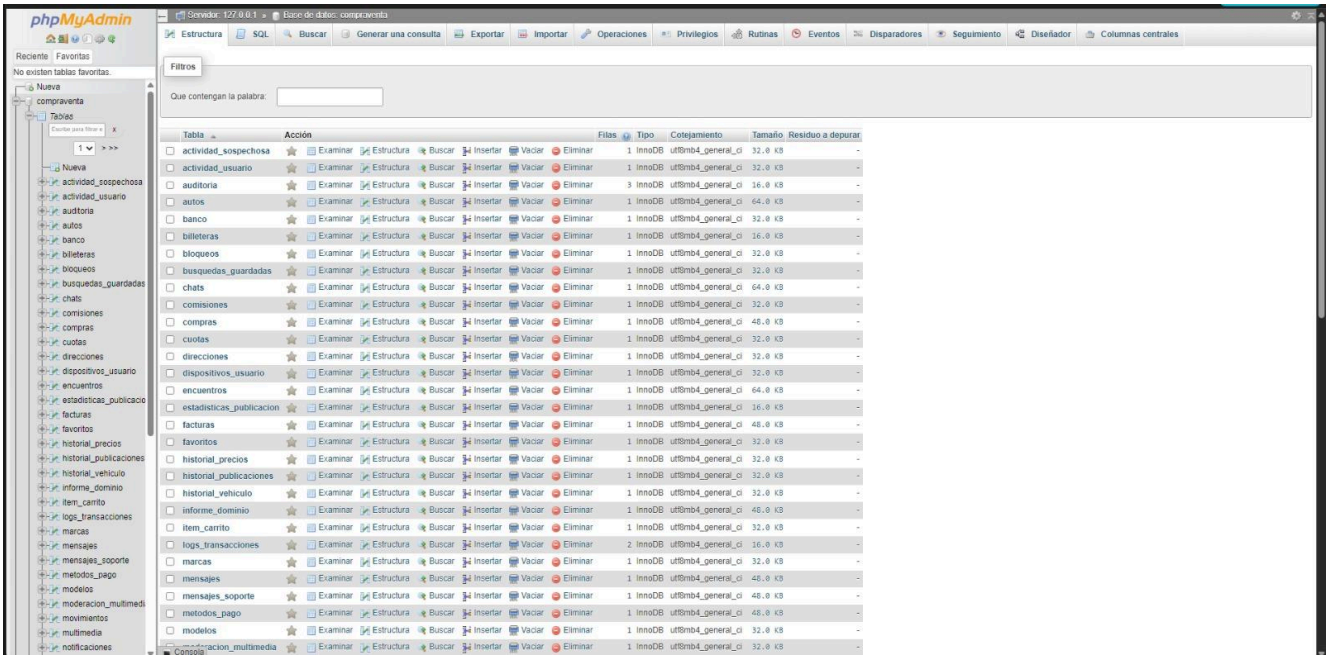
3.1.3. Importación de compraventa.sql e integracion.sql

1. Creación de la base de datos: En el administrador de base de datos (por ejemplo, phpMyAdmin en XAMPP), crear una nueva base de datos llamada compraventa.
2. Carga de la estructura e información base: Importar el archivo

`Base_De_Datos_CarFind/compraventa.sql` dentro de la base de datos recién creada. 3. Carga automática de datos para el frontend: No se requiere ejecutar scripts adicionales manualmente; al iniciar el backend, el servidor ejecuta automáticamente el archivo `Backend_CarFind/carfind/src/main/resources/db/integracion.sql`. Este script agrega los elementos requeridos por el frontend (foto de perfil por defecto, tipos de reporte, respuestas administrativas, informes y el rol ADMIN), y está diseñado para ejecutarse múltiples veces sin generar conflictos.

3.1.4. Verificación de tablas y datos iniciales

Para verificar la correcta inicialización, se debe ingresar a phpMyAdmin o realizar una consulta en la base de datos compraventa comprobando la presencia de las tablas principales del sistema, como usuarios, sesiones, usuario_roles, publicaciones, autos, modelos, marcas, multimedia, direcciones, favoritos, chats, mensajes y reportes.



Verificación de la estructura de tablas importadas en phpMyAdmin.

3.2. Configuración y ejecución del backend

3.2.1. Configuración de la conexión con la base de datos

La conexión entre el servidor Java / Spring Boot y la base de datos se gestiona mediante el archivo de propiedades del proyecto:

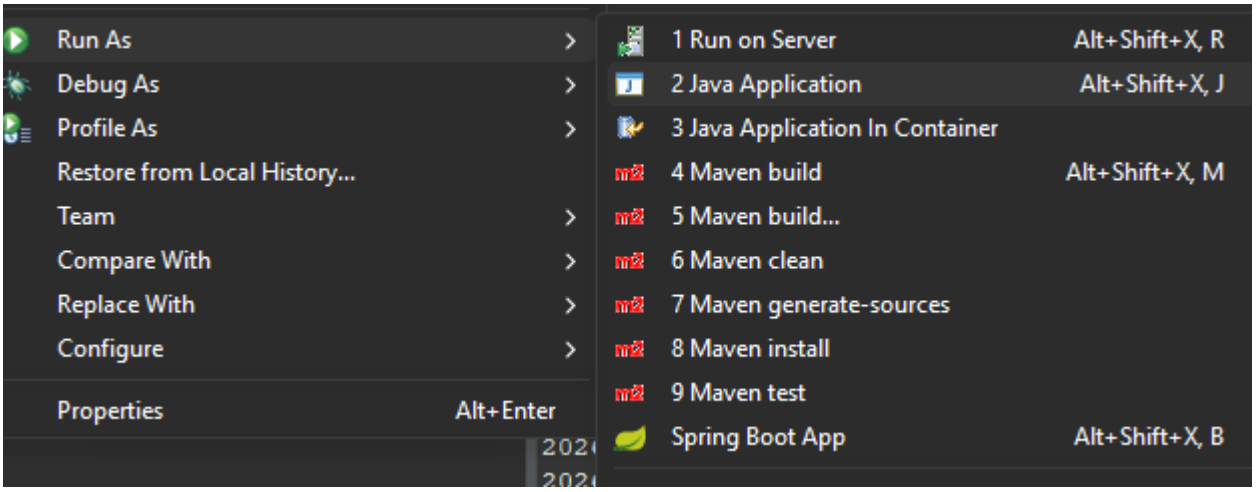
Ubicación: `Backend_CarFind/carfind/src/main/resources/application.properties`. Parámetros configurados: Dirección de la base de datos en el puerto 3307, usuario root sin contraseña y nombre de base de datos compraventa.

3.2.2. Ejecución mediante Eclipse o consola Maven

Existen dos modalidades para levantar el servidor backend en Spring Boot:

Opción A: Desde IDE Eclipse:

1. Abrir el proyecto Backend_CarFind/carfind.
2. Hacer clic derecho sobre el archivo ejecutable CarfindApplication.java.
3. Seleccionar Run As Java Application (o Spring Boot App).



Opción B: Desde la Consola de Comandos (Maven):

1. Abrir la terminal y navegar al directorio del backend:

   ```
   cd Backend_CarFind/carfind
   ```

2. Ejecutar el comando de arranque:

   ```
   mvn spring-boot:run
   ```

3.2.3. Configuración de rutas y almacenamiento de archivos

Puerto y URL predeterminada: El servidor queda escuchando en `http://localhost:8080`.

Estructura de endpoints:

La API utilizada por la interfaz frontend está mapeada bajo la ruta `/api/web/**` (perteneciente al paquete Java com.back.carfind.web).

Las operaciones CRUD originales de mantenimiento continúan operativas bajo la ruta `/api/**`

Servidor de archivos multimedia: Las fotografías cargadas por los usuarios se almacenan en el directorio local `uploads/` y se sirven a través del endpoint `/uploads/**`

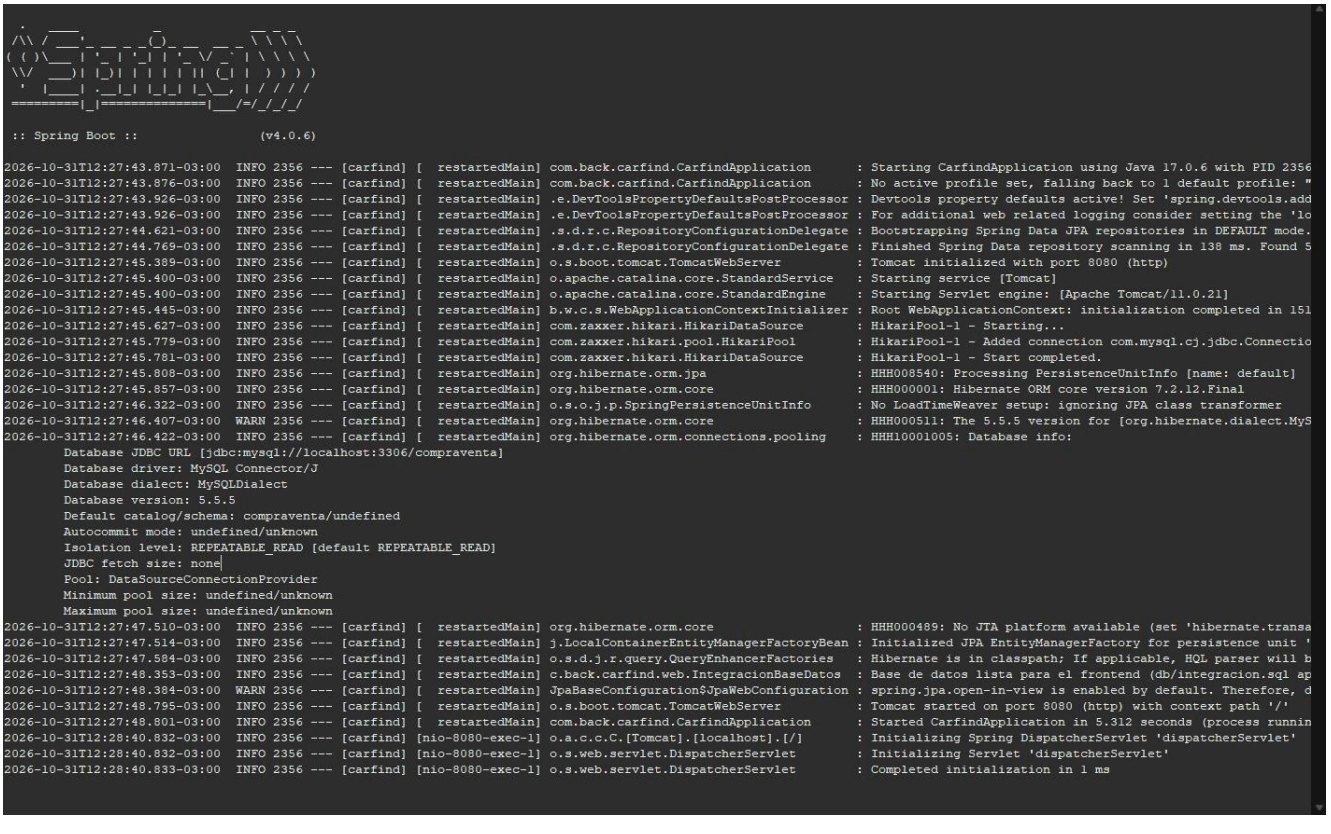
Despliegue alternativo como WAR: Si el backend se despliega como un archivo WAR dentro de un

servidor Apache Tomcat utilizando un context path diferente, se debe actualizar la variable BACKEND_URL en el archivo `Frontend_CarFind/vite.config.js`.

3.2.4. Verificación del servidor y la API

Para comprobar que el servicio está respondiendo correctamente:

1. Confirmar en la consola o terminal la salida de arranque sin excepciones en Spring Boot. 2. Comprobar la accesibilidad del puerto 8080 realizando una petición GET a las rutas públicas de la API (`http://localhost:8080/api/web/...`) o consultando la interfaz interactiva de Swagger UI.



```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v4.0.6)

2026-10-31T12:27:43.871-03:00  INFO 2356 --- [carfind] [  restartedMain] com.back.carfind.CarfindApplication      : Starting CarfindApplication using Java 17.0.6 with PID 2356
2026-10-31T12:27:43.876-03:00  INFO 2356 --- [carfind] [  restartedMain] com.back.carfind.CarfindApplication      : No active profile set, falling back to 1 default profile: "
2026-10-31T12:27:43.926-03:00  INFO 2356 --- [carfind] [  restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : Devtools property defaults active! Set 'spring.devtools.add
2026-10-31T12:27:43.926-03:00  INFO 2356 --- [carfind] [  restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : For additional web related logging consider setting the 'lo
2026-10-31T12:27:44.621-03:00  INFO 2356 --- [carfind] [  restartedMain] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-10-31T12:27:44.769-03:00  INFO 2356 --- [carfind] [  restartedMain] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 138 ms. Found 5
2026-10-31T12:27:45.389-03:00  INFO 2356 --- [carfind] [  restartedMain] o.s.boot.tomcat.TomcatWebServer          : Tomcat initialized with port 8080 (http)
2026-10-31T12:27:45.400-03:00  INFO 2356 --- [carfind] [  restartedMain] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2026-10-31T12:27:45.400-03:00  INFO 2356 --- [carfind] [  restartedMain] o.apache.catalina.core.StandardEngine    : Starting Servlet engine: [Apache Tomcat/11.0.21]
2026-10-31T12:27:45.445-03:00  INFO 2356 --- [carfind] [  restartedMain] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 151
2026-10-31T12:27:45.627-03:00  INFO 2356 --- [carfind] [  restartedMain] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Starting...
2026-10-31T12:27:45.779-03:00  INFO 2356 --- [carfind] [  restartedMain] com.zaxxer.hikari.pool.HikariPool        : HikariPool-1 - Added connection com.mysql.cj.jdbc.Connectio
2026-10-31T12:27:45.781-03:00  INFO 2356 --- [carfind] [  restartedMain] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Start completed.
2026-10-31T12:27:45.808-03:00  INFO 2356 --- [carfind] [  restartedMain] org.hibernate.orm.jpa                    : HHH008540: Processing PersistenceUnitInfo [name: default]
2026-10-31T12:27:45.857-03:00  INFO 2356 --- [carfind] [  restartedMain] org.hibernate.orm.core                   : HHH000001: Hibernate ORM core version 7.2.12.Final
2026-10-31T12:27:46.322-03:00  INFO 2356 --- [carfind] [  restartedMain] o.s.o.j.p.SpringPersistenceUnitInfo      : No LoadTimeWeaver setup: ignoring JPA class transformer
2026-10-31T12:27:46.407-03:00  WARN 2356 --- [carfind] [  restartedMain] org.hibernate.orm.core                   : HHH000511: The 5.5.5 version for [org.hibernate.dialect.MyS
2026-10-31T12:27:46.422-03:00  INFO 2356 --- [carfind] [  restartedMain] org.hibernate.orm.connections.pooling    : HHH10001005: Database info:
       Database JDBC URL [jdbc:mysql://localhost:3306/compraventa]
       Database driver: MySQL Connector/J
       Database dialect: MySQLDialect
       Database version: 5.5.5
       Default catalog/schema: compraventa/undefined
       Autocommit mode: undefined/unknown
       Isolation level: REPEATABLE_READ [default REPEATABLE_READ]
       JDBC fetch size: none
       Pool: DataSourceConnectionProvider
       Minimum pool size: undefined/unknown
       Maximum pool size: undefined/unknown
2026-10-31T12:27:47.510-03:00  INFO 2356 --- [carfind] [  restartedMain] org.hibernate.orm.core                   : HHH000489: No JTA platform available (set 'hibernate.transa
2026-10-31T12:27:47.514-03:00  INFO 2356 --- [carfind] [  restartedMain] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit '
2026-10-31T12:27:47.584-03:00  INFO 2356 --- [carfind] [  restartedMain] o.s.d.j.r.query.QueryEnhancerFactories   : Hibernate is in classpath; If applicable, HQL parser will b
2026-10-31T12:27:48.353-03:00  INFO 2356 --- [carfind] [  restartedMain] c.back.carfind.web.IntegracionBaseDatos  : Base de datos lista para el frontend (db/integracion.sql ap
2026-10-31T12:27:48.384-03:00  WARN 2356 --- [carfind] [  restartedMain] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, d
2026-10-31T12:27:48.795-03:00  INFO 2356 --- [carfind] [  restartedMain] o.s.boot.tomcat.TomcatWebServer          : Tomcat started on port 8080 (http) with context path '/'
2026-10-31T12:27:48.801-03:00  INFO 2356 --- [carfind] [  restartedMain] com.back.carfind.CarfindApplication      : Started CarfindApplication in 5.312 seconds (process runnin
2026-10-31T12:28:40.832-03:00  INFO 2356 --- [carfind] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-10-31T12:28:40.832-03:00  INFO 2356 --- [carfind] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet        : Initializing Servlet 'dispatcherServlet'
2026-10-31T12:28:40.833-03:00  INFO 2356 --- [carfind] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet        : Completed initialization in 1 ms
```

Consola de ejecución del backend Spring Boot iniciado en puerto 8080.

3.3. Configuración y ejecución del frontend

3.3.1. Instalación de dependencias

Para preparar la aplicación cliente en React:

1. Abrir la terminal de comandos y situarse en la carpeta del frontend:

   ```
   cd Frontend_CarFind
   ```

2. Instalar las dependencias de Node.js:

   ```
   npm install
   ```

3.3.2. Configuración de la URL del backend

La integración del frontend con el servidor se gestiona a través del archivo de variables de entorno `.env` ubicado en la raíz de Frontend_CarFind:

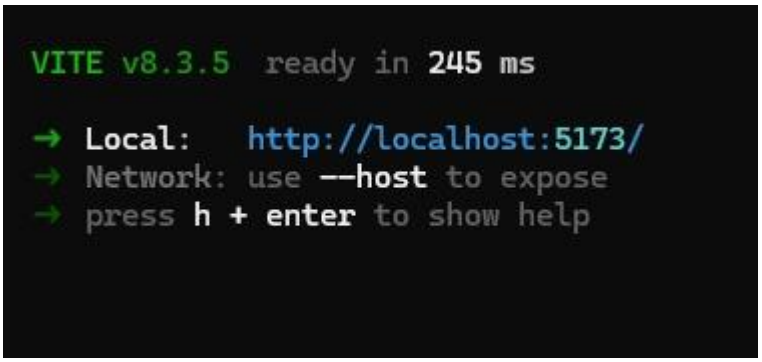
`VITE_API_URL=/api/web`: Establece la ruta base para realizar las peticiones a la API. Configuración del Proxy de Vite: En `vite.config.js`, Vite redirige automáticamente las llamadas dirigidas a `/api/**` y `/uploads/**` desde el puerto local del frontend (5173) hacia el puerto del backend (8080).

3.3.3. Ejecución del servidor de desarrollo Vite

1. Iniciar el servidor de desarrollo local ejecutando:

   ```
   npm run dev
   ```

2. Abrir el navegador e ingresar a la URL `http://localhost:5173`.



Arranque del servidor de desarrollo Vite para la aplicación en React.

3.3.4. Configuración de los modos de integración: API real y Mock

La variable `VITE_USE_MOCK` en el archivo `.env` permite alternar el origen de los datos de la aplicación:

Modo API Real (VITE_USE_MOCK=false): Configuración predeterminada. Todas las pantallas del frontend leen y persisten datos en la base de datos MySQL/MariaDB a través del backend en Spring Boot.

Modo Datos de Prueba (VITE_USE_MOCK=true): Activa una capa de datos simulados (mock data) en el cliente, permitiendo probar la interfaz y el flujo visual sin necesidad de tener el backend o la base de datos en ejecución.

4. Validación e integración del sistema

La información sobre las credenciales de prueba (juan.perez@mail.com y maria.gomez@mail.com) y la estrategia de Data Seeding con aproximadamente 50 registros de vehículos para la validación de búsquedas y filtros está plenamente respaldada por las fuentes.

4.1. Cuentas y roles de prueba

Para llevar a cabo las pruebas de integración del sistema y validar la correcta asignación de permisos y niveles de acceso, la base de datos cuenta con cuentas de prueba preconfiguradas:

Correo electrónico	Contraseña	Rol asignado	Permisos y alcance
juan.perez@mail.com	Carfind123	Usuario	Acceso estándar para explorar el catálogo, guardar favoritos, enviar mensajes y publicar vehículos.
maria.gomez@mail.com	Carfind123	Administradora	Acceso a funciones globales de moderación, visualización del botón "Administración" y acceso al panel /admin.

Creación de nuevos usuarios: Adicionalmente, se pueden registrar nuevas cuentas desde la pantalla de "Registrarse" para verificar el flujo completo de alta de usuarios.

Estrategia de Carga y Pruebas de Datos (Data Seeding): En lugar de ejecutar scripts masivos sintéticos mediante herramientas como JMeter o Postman, la estrategia de validación de carga y rendimiento de la base de datos se basó en un proceso de Data Seeding. Se preparó e ingresó un conjunto representativo de aproximadamente 50 registros de vehículos con variedad de marcas, modelos, años, precios y kilometrajes. Esto permitió simular escenarios reales de uso, evaluar la precisión del buscador y verificar los tiempos de respuesta de los filtros combinados sin sobrecargar el entorno.

4.2. Verificación de la comunicación entre frontend y backend

La comunicación entre la interfaz en React + Vite (puerto 5173) y el servidor Spring Boot (puerto 8080) se valida mediante la correcta canalización de solicitudes HTTP:

1. Configuración de Proxy local: El archivo `vite.config.js` del frontend actúa como proxy, redirigiendo todas las llamadas hacia las rutas `/api/**` y `/uploads/**` desde el puerto 5173 al backend en `http://localhost:8080`.
2. Habilitación de CORS: El servidor Spring Boot mantiene habilitada la política de intercambio de recursos de origen cruzado (CORS) para responder adecuadamente a las solicitudes origen `http://localhost:5173`.
3. Manejo de Autenticación y Sesiones (JWT): Al autenticarse mediante `auth/login`, el backend genera un token JWT que el frontend almacena en `localStorage` para mantener la sesión activa entre pantallas.
4. Verificación de Variables de Entorno y Configuración HTTP: En `Frontend_CarFind/.env`, la variable `VITE_USE_MOCK=false` confirma que la aplicación se comunica directamente con la API real en lugar del entorno de datos simulados.

Para garantizar una navegación fluida, rápida y prevenir que la interfaz se congele ante problemas de red, se configuran los siguientes parámetros en el cliente HTTP (Axios / Fetch):

Tiempo de espera estándar (Timeout global): 5.000 ms (5 segundos) para solicitudes de lectura y formularios (autenticación, catálogo, búsquedas y perfil). Si el servidor no responde dentro de este lapso, la petición se cancela automáticamente y se notifica al usuario.

Timeout extendido para multimedia: 15.000 ms (15 segundos) para solicitudes de tipo multipart que involucren la carga y procesamiento de imágenes de vehículos (`/uploads/**`). Estrategia de reintentos automáticos (Retry Policy):

Número máximo de reintentos: 2 intentos adicionales con un intervalo de espera exponencial (500 ms y 1.000 ms).

Criterio de aplicación: Se activa únicamente ante interrupciones de red o errores de servidor 5xx (502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout).

Exclusión de reintentos: Se excluyen los errores de cliente 4xx (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found), garantizando que las validaciones de usuario e

inicio de sesión respondan en tiempo real.

4.3. Verificación de endpoints y respuestas de la API

La arquitectura del backend expone los servicios web agrupados principalmente en el paquete `com.back.carfind.web` bajo la ruta `/api/web/**`, manteniendo compatibles los endpoints CRUD originales en `/api/**`.

Mapeo entre Pantallas del Frontend, Endpoints y Tablas Relacionales:

Pantalla	Endpoints (/api/web/...)	Tablas principales impactadas
Login / Registro	auth/login, auth/registro, auth/sesion, auth/logout	usuarios, sesiones, usuario_roles
Inicio (filtros y buscador)	publicaciones, publicaciones/filtros	publicaciones, autos, modelos, marcas, multimedia
Detalle de publicación	publicaciones/:id	publicaciones, autos, modelos, marcas, multimedia, direcciones
Crear publicación	POST publicaciones (multipart con fotos)	autos, publicaciones, multimedia
Wishlist (Favoritos)	wishlist	favoritos
Perfil / Configuración / Mensajes / Reportes	perfil, perfil/foto, usuarios/:id, configuracion, conversaciones, reportes, publicaciones/:id/reporte	usuarios, direcciones, preferencias_usuario, chats, mensajes, reportes
Panel de administración	admin/**	reportes, respuestas_reporte, informes_admin

Documentación Interactiva de la API: La API se encuentra documentada y se puede probar directamente

desde el navegador mediante Swagger UI (EPIC-12), además de contar con la plantilla de referencia en hoja de cálculo/PDF (API CARFIND).

4.4. Comprobación de operaciones sobre la base de datos

Para asegurar la persistencia e integridad relacional en la base de datos MySQL/MariaDB (compraventa en el puerto 3307):

1. Carga automática en el arranque: Al iniciar el backend Spring Boot, se ejecuta automáticamente el script `Backend_CarFind/carfind/src/main/resources/db/integracion.sql`. Este script inserta la foto de perfil por defecto, los tipos de reporte, el rol ADMIN, las respuestas administrativas y los informes requeridos por el cliente sin romper registros existentes.
2. Servicio de archivos multimedia (uploads/): Las imágenes de vehículos y perfiles subidas mediante formularios multipart se guardan físicamente en la carpeta `Backend_CarFind/carfind/uploads/` y son expuestas por el servidor en `/uploads/**`.
3. Verificación de consistencia: Se comprueba la correcta inserción en las tablas publicaciones, autos y multimedia tras crear o editar una publicación.

4.5. Pruebas funcionales de integración (de extremo a extremo)

Para validar la integración completa del sistema (la comunicación entre React, Spring Boot y MySQL/MariaDB), se realiza una verificación manual de los principales recorridos de usuario:

Flujo de usuario comprador:

1. Registro o inicio de sesión con las credenciales de un usuario (juan.perez@mail.com). 2. Búsqueda y filtrado dinámico de vehículos desde la pantalla de Inicio.
3. Acceso a la Ficha Técnica (/publicaciones/:id) y guardado del vehículo en la Lista de Favoritos (/wishlist).

4. Envío de consultas o inicio de una conversación con el vendedor (/conversaciones/**).

Flujo de usuario vendedor:

1. Acceso a la pantalla "Subir Publicación".

2. Carga de un nuevo vehículo, completando sus especificaciones técnicas y adjuntando imágenes. 3. Verificación de que los datos se guarden correctamente en las tablas autos y publicaciones, y de que las fotografías se almacenen físicamente en el directorio uploads/.

Flujo de moderación administrativa:

1. Generación de un reporte desde la vista de una publicación (/publicaciones/:id/reportes). 2. Inicio de sesión con una cuenta que tenga el rol de Administrador (maria.gomez@mail.com). 3. Ingreso al panel de administración (/admin) para revisar y gestionar los reportes e informes recibidos.

Nota sobre el entorno: Una vez verificados satisfactoriamente todos los criterios de aceptación mediante estos recorridos integrados de principio a fin, no se considera necesaria la ejecución de pruebas automatizadas adicionales, como Cypress o Selenium, para la puesta en marcha de esta versión.

5. Despliegue en infraestructura Cloud

5.1. Configuración de la base de datos gestionada (Aiven)

La infraestructura en la nube contempla el uso de Aiven como servicio PaaS gestionado para alojar y administrar la base de datos relacional MySQL del sistema CarFind.

> Creación del servicio en Aiven: Configuración e inicialización de una instancia de base de datos MySQL en la consola de Aiven.
>
> Datos de conexión: Los parámetros de acceso se configuran mediante variables de entorno en la plataforma de alojamiento del backend (`SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME` y `SPRING_DATASOURCE_PASSWORD`), utilizando conexiones seguras SSL mediante cadenas del tipo `jdbc:mysql://<aiven-host>:<port>/compraventa?sslmode=REQUIRED`.

Migración de esquemas y datos iniciales:

Adaptación para la nube (preparar_para_nube.py): Dado que Aiven exige claves primarias en la creación de cada tabla (sql_require_primary_key) y rechaza la cláusula DEFINER=root@localhost, se ejecuta previamente el script en Python preparar_para_nube.py sobre compraventa.sql. Este script genera el archivo compraventa_nube.sql, reubicando las claves primarias directamente dentro de cada sentencia CREATE TABLE, eliminando las tablas stand-in temporales creadas por phpMyAdmin para las vistas y reemplazando la seguridad por SQL SECURITY INVOKER. Ejecución remota: La importación del archivo compraventa_nube.sql en la instancia de Aiven se realiza mediante clientes de administración como DBeaver, MySQL Workbench o la línea de comandos CLI de MySQL.

Carga de datos de integración: Al conectar el servidor backend por primera vez, el proceso de arranque ejecuta automáticamente el script `integracion.sql`. Esto garantiza la carga de la foto de perfil por defecto, el rol ADMIN, los tipos de reporte, las respuestas administrativas e informes sin generar conflictos con registros existentes.

5.2. Configuración y despliegue del backend (Render)

El servidor Spring Boot se despliega en la plataforma Cloud PaaS Render, encargada de compilar, ejecutar la API REST y procesar la lógica de negocio.

Estrategia de despliegue:

Vinculación del repositorio Git de la aplicación backend (Backend_CarFind/carfind) con el servicio Web Service de Render para habilitar el despliegue continuo (CD) tras cada actualización de código.

Entorno de ejecución: Se utiliza el entorno nativo Java/Maven configurado con la versión de Java JDK 17 (o la ejecución mediante un contenedor Dockerfile según las políticas del equipo).

Configuración del servicio en Render:

Puerto de escucha: El servicio se configura para tomar de forma dinámica el puerto expuesto en producción por la variable de entorno PORT de Render.

Comando de compilación (Build Command): `cd Backend_CarFind/carfind && mvn clean package -DskipTests`

Comando de arranque (Start Command): `java -jar Backend_CarFind/carfind/target/carfind`

0.0.1-SNAPSHOT.jar

Almacenamiento de archivos multimedia (uploads/):

Las imágenes cargadas por los usuarios en las publicaciones y perfiles se guardan en el directorio /carfind/uploads/ y se sirven a través del endpoint /uploads/**.

Disco Persistente (Persistent Disk): En el panel de Render se configura un volumen persistente montado en la ruta /carfind/uploads/ para asegurar que las fotografías almacenadas no se eliminen al reiniciar la instancia o realizar nuevos despliegues.

5.3. Configuración y despliegue del frontend (Vercel)

La aplicación React + Vite se aloja en la plataforma Vercel para su compilación y distribución global optimizada.

Creación del proyecto en Vercel:

Importación del repositorio Git desde el directorio Frontend_CarFind.

Framework Preset: Vite.

Comando de compilación (Build Command): npm run build

Directorio de salida (Output Directory): dist

Ruteo y Proxy de solicitudes (vercel.json): Para canalizar las peticiones a la API y archivos multimedia hacia Render, así como para asegurar la navegación de la SPA en React, se incluye el archivo vercel.json en la raíz del proyecto:

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "rewrites": [
    { "source": "/api/:path*", "destination": "https://TU
BACKEND.onrender.com/api/:path*" },
    { "source": "/uploads/:path*", "destination": "https://TU
BACKEND.onrender.com/uploads/:path*" },
```

```
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

Explicación de las reglas en vercel.json:

`/api/:path*`: Redirige de forma transparente todas las llamadas a la API hacia la URL pública del backend en Render.

`/uploads/:path*`: Redirige las solicitudes de imágenes hacia el servidor backend en Render donde está montado el disco persistente.

`/(.*) -> /index.html`: Regla comodín que reescribe todas las rutas restantes hacia el punto de entrada principal de React (index.html), permitiendo que el router del cliente controle la navegación sin producir errores 404 al recargar pantallas en el navegador.

5.4. Configuración de variables de entorno y conexiones entre servicios

Para vincular adecuadamente los tres servicios en la nube:

1. Variables de Entorno en Render (Backend Spring Boot):

   `SPRING_DATASOURCE_URL`: Cadena de conexión JDBC hacia la instancia MySQL alojada en Aiven (ej. `jdbc:mysql://<host_aiven>:<port_aiven>/compraventa`).

   `SPRING_DATASOURCE_USERNAME`: Usuario asignado en Aiven.

   `SPRING_DATASOURCE_PASSWORD`: Contraseña asignada en Aiven.

   `JWT_SECRET`: Clave secreta alfanumérica configurada en las variables de entorno de Render para la firma, emisión y validación de los tokens de sesión JWT.

2. Variables de Entorno en Vercel (Frontend React + Vite):

   `VITE_USE_MOCK=false`: Desactiva los datos simulados de prueba para operar con la base de datos real.

   `VITE_API_URL`: URL pública del backend en Render.

5.5. Verificación de la aplicación desplegada

Una vez completado el despliegue de los tres servicios Cloud, se llevan a cabo las pruebas de verificación en el entorno de producción:

1. Conectividad Base de Datos - Backend: Verificar en los logs del servidor Spring Boot en Render que la conexión JDBC con Aiven se establece correctamente y que el script integracion.sql se ejecuta sin errores.
2. Conectividad Frontend - Backend: Ingresar al dominio de Vercel y comprobar que la pantalla de inicio cargue las publicaciones existentes desde la base de datos a través de la API en Render. 3. Autenticación y Sesiones en Producción: Iniciar sesión con la cuenta de administrador (maria.gomez@mail.com) o usuario (juan.perez@mail.com) y confirmar la persistencia del token JWT en el navegador.
4. Carga y visualización multimedia: Crear una nueva publicación con imágenes y validar que las fotos se almacenen físicamente en el servidor backend y se muestren correctamente desde la ruta `/uploads/**`.
5. Verificación de API interactiva: Comprobar el acceso a la documentación interactiva de Swagger UI alojada en el backend en producción.

6. Anexos y documentación técnica

6.1. Comandos utilizados para instalación y ejecución

A continuación se resumen los comandos fundamentales utilizados para instalar dependencias, compilar y ejecutar cada una de las capas del sistema CarFind en el entorno local:

Base de datos (MySQL / MariaDB en XAMPP):

Inicialización del servicio MySQL a través del panel de control de XAMPP asegurando el puerto 3307. Creación de la base de datos e importación de la estructura inicial mediante terminal o phpMyAdmin:

```
mysql -u root -P 3307 -e "CREATE DATABASE compraventa;"
```

```
mysql -u root -P 3307 compraventa < Base_De_Datos_CarFind/compraventa.sql
```

Backend (Spring Boot / Java):

Navegación al directorio ejecutable y arranque del servidor en el puerto 8080 mediante consola Maven:

```
cd Backend_CarFind/carfind
mvn spring-boot:run
```

Ejecución alternativa desde IDE Eclipse: Clic derecho sobre CarfindApplication.java → Run As Java Application (o Spring Boot App).

Frontend (React + Vite):

Instalación de paquetes e inicio del servidor de desarrollo local en el puerto 5173:

```
cd Frontend_CarFind
npm install
npm run dev
```

Generación del empaquetado optimizado para entorno de producción:

```
npm run build
```

Despliegue manual por consola CLI (alternativo a Git):

En caso de requerir un despliegue directo por terminal sin integración continua: **Vercel**: Ejecutar `npm i -g vercel` seguido del comando `vercel --prod`.

Render: Ejecutar `git push render main` o desencadenar el deploy mediante la URL del Webhook oficial de Render.

6.2. Archivos de configuración de referencia

Para garantizar la coherencia en la integración cliente-servidor y el almacenamiento de datos, se destacan los siguientes archivos clave de configuración:

- Configuración de base de datos en el Backend (application.properties):
  - Ubicación: `Backend_CarFind/carfind/src/main/resources/application.properties`. Parámetros principales:

```
server.port=8080
spring.datasource.url=jdbc:mysql://localhost:3307/compraventa
spring.datasource.username=root
spring.datasource.password=
```

- Script de inicialización automática (integracion.sql):
  - Ubicación: `Backend_CarFind/carfind/src/main/resources/db/integracion.sql`.
  - Función: Inserción idempotente de registros iniciales requeridos por el cliente (foto de perfil por defecto, tipos de reporte, rol ADMIN, respuestas administrativas e informes) al arrancar el backend.
- Variables de entorno del Frontend (.env):
  - Ubicación: `Frontend_CarFind/.env`.
  - Parámetros principales:

```
VITE_USE_MOCK=false
VITE_API_URL=/api/web
```

- Configuración de Proxy local (vite.config.js):
  - Ubicación: `Frontend_CarFind/vite.config.js`.
  - Función: Canaliza las peticiones dirigidas a `/api/**` y `/uploads/**` desde el puerto 5173 hacia el puerto 8080 del backend (o hacia la variable BACKEND_URL en despliegues con context path en Tomcat).

Plantilla de configuración de Vercel (`vercel.json`):

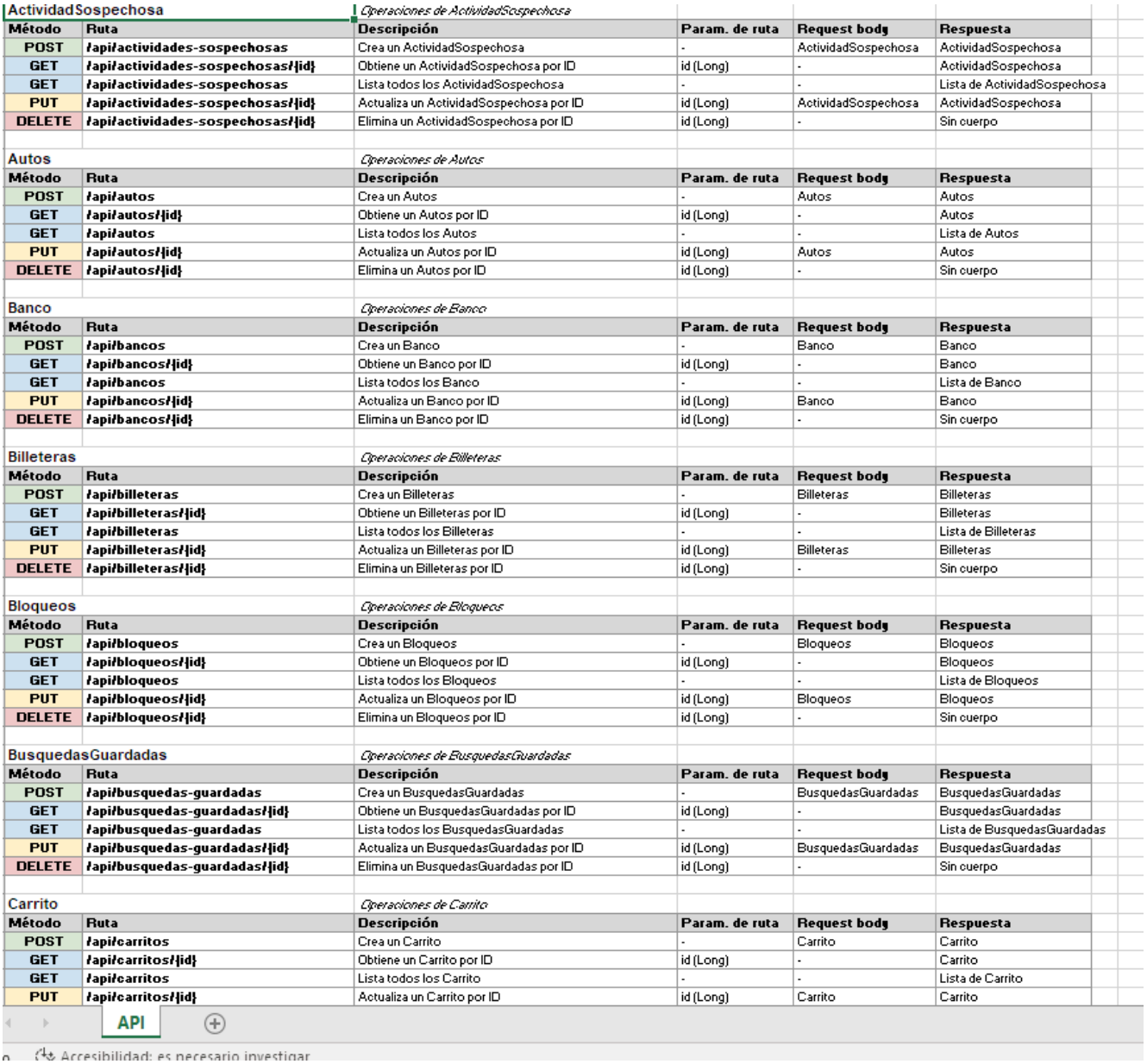
Ubicado en la raíz de `Frontend_CarFind/`, define las reescrituras de rutas hacia Render y resuelve el ruteo de la SPA:

```
{ "$schema": "https://openapi.vercel.sh/vercel.json", "rewrites": [ { "source": "/api/:path*", "destination": "https://TU-BACKEND.onrender.com/api/:path*" }, { "source": "/uploads/:path*", "destination": "https://TU-BACKEND.onrender.com/uploads/:path*" }, { "source": "/((?!api|uploads|.*\\.).*)", "destination": "/index.html" } ] }
```

6.3. Documentación de la API (Swagger)

El sistema integra la especificación Swagger UI (incorporada formalmente en la Épica EPIC-12 del Backlog), lo que brinda una interfaz gráfica para consultar y validar los servicios REST expuestos por el backend.

- Acceso a la interfaz interactiva: En entorno local, Swagger UI se encuentra disponible ingresando a `http://localhost:8080/swagger-ui/index.html` (o `http://localhost:8080/swagger-ui.html`). Utilidad y alcance de pruebas:
  - Permite inspeccionar de forma organizada los controladores REST y sus métodos mapeados. Ofrece una consola interactiva para realizar solicitudes HTTP (GET, POST, PUT, DELETE) enviando payloads en formato JSON y evaluando las respuestas en tiempo real.
  - Valida las estructuras de datos y los códigos de respuesta HTTP (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error).



ActividadSospechosa		*Operaciones de ActividadSospechosa*			
Método	**Ruta**	**Descripción**	**Param. de ruta**	**Request body**	**Respuesta**
POST	/api/actividades-sospechosas	Crea un ActividadSospechosa	-	ActividadSospechosa	ActividadSospechosa
GET	/api/actividades-sospechosas/{id}	Obtiene un ActividadSospechosa por ID	id (Long)	-	ActividadSospechosa
GET	/api/actividades-sospechosas	Lista todos los ActividadSospechosa	-	-	Lista de ActividadSospechosa
PUT	/api/actividades-sospechosas/{id}	Actualiza un ActividadSospechosa por ID	id (Long)	ActividadSospechosa	ActividadSospechosa
DELETE	/api/actividades-sospechosas/{id}	Elimina un ActividadSospechosa por ID	id (Long)	-	Sin cuerpo
Autos		*Operaciones de Autos*			
Método	**Ruta**	**Descripción**	**Param. de ruta**	**Request body**	**Respuesta**
POST	/api/autos	Crea un Autos	-	Autos	Autos
GET	/api/autos/{id}	Obtiene un Autos por ID	id (Long)	-	Autos
GET	/api/autos	Lista todos los Autos	-	-	Lista de Autos
PUT	/api/autos/{id}	Actualiza un Autos por ID	id (Long)	Autos	Autos
DELETE	/api/autos/{id}	Elimina un Autos por ID	id (Long)	-	Sin cuerpo
Banco		*Operaciones de Banco*			
Método	**Ruta**	**Descripción**	**Param. de ruta**	**Request body**	**Respuesta**
POST	/api/bancos	Crea un Banco	-	Banco	Banco
GET	/api/bancos/{id}	Obtiene un Banco por ID	id (Long)	-	Banco
GET	/api/bancos	Lista todos los Banco	-	-	Lista de Banco
PUT	/api/bancos/{id}	Actualiza un Banco por ID	id (Long)	Banco	Banco
DELETE	/api/bancos/{id}	Elimina un Banco por ID	id (Long)	-	Sin cuerpo
Billeteras		*Operaciones de Billeteras*			
Método	**Ruta**	**Descripción**	**Param. de ruta**	**Request body**	**Respuesta**
POST	/api/billeteras	Crea un Billeteras	-	Billeteras	Billeteras
GET	/api/billeteras/{id}	Obtiene un Billeteras por ID	id (Long)	-	Billeteras
GET	/api/billeteras	Lista todos los Billeteras	-	-	Lista de Billeteras
PUT	/api/billeteras/{id}	Actualiza un Billeteras por ID	id (Long)	Billeteras	Billeteras
DELETE	/api/billeteras/{id}	Elimina un Billeteras por ID	id (Long)	-	Sin cuerpo
Bloqueos		*Operaciones de Bloqueos*			
Método	**Ruta**	**Descripción**	**Param. de ruta**	**Request body**	**Respuesta**
POST	/api/bloqueos	Crea un Bloqueos	-	Bloqueos	Bloqueos
GET	/api/bloqueos/{id}	Obtiene un Bloqueos por ID	id (Long)	-	Bloqueos
GET	/api/bloqueos	Lista todos los Bloqueos	-	-	Lista de Bloqueos
PUT	/api/bloqueos/{id}	Actualiza un Bloqueos por ID	id (Long)	Bloqueos	Bloqueos
DELETE	/api/bloqueos/{id}	Elimina un Bloqueos por ID	id (Long)	-	Sin cuerpo
BusquedasGuardadas		*Operaciones de BusquedasGuardadas*			
Método	**Ruta**	**Descripción**	**Param. de ruta**	**Request body**	**Respuesta**
POST	/api/busquedas-guardadas	Crea un BusquedasGuardadas	-	BusquedasGuardadas	BusquedasGuardadas
GET	/api/busquedas-guardadas/{id}	Obtiene un BusquedasGuardadas por ID	id (Long)	-	BusquedasGuardadas
GET	/api/busquedas-guardadas	Lista todos los BusquedasGuardadas	-	-	Lista de BusquedasGuardadas
PUT	/api/busquedas-guardadas/{id}	Actualiza un BusquedasGuardadas por ID	id (Long)	BusquedasGuardadas	BusquedasGuardadas
DELETE	/api/busquedas-guardadas/{id}	Elimina un BusquedasGuardadas por ID	id (Long)	-	Sin cuerpo
Carrito		*Operaciones de Carrito*			
Método	**Ruta**	**Descripción**	**Param. de ruta**	**Request body**	**Respuesta**
POST	/api/carritos	Crea un Carrito	-	Carrito	Carrito
GET	/api/carritos/{id}	Obtiene un Carrito por ID	id (Long)	-	Carrito
GET	/api/carritos	Lista todos los Carrito	-	-	Lista de Carrito
PUT	/api/carritos/{id}	Actualiza un Carrito por ID	id (Long)	Carrito	Carrito

API

Accesibilidad: es necesario investigar

6.4. Referencia al documento API_CARFIND

Como soporte estructurado a la interfaz interactiva de Swagger, la arquitectura cuenta con la documentación técnica consolidada en el archivo api_swagger.xlsx API CARFIND.pdf.

Descripción del documento: Establece la especificación formal de la API, detallando las operaciones CRUD (Create, Read, Update, Delete) sobre las entidades del modelo relacional. Estructura del mapeo:

Métodos HTTP y Rutas: Asignación de verbos (POST, GET, PUT, DELETE) a las rutas bajo el prefijo `/api/{entidad}`.

Parámetros de ruta: Identificación de identificadores obligatorios, como id (Long). Cuerpos de solicitud (Request Body): Definición del objeto JSON requerido para operaciones de creación o

actualización.

Respuestas del servidor: Especificación del tipo de retorno (entidad individual, lista de entidades o respuesta vacía 204 No Content).

Entidades y módulos documentados:

Usuarios y Autenticación: usuarios, sesiones, roles, direcciones, dispositivos-usuario. Catálogo y Vehículos: autos, publicaciones, marcas, modelos, tipos-combustible, transmisiones, multimedia, historial-vehiculos, informes-dominio.

Comercialización y Pagos: favoritos, carritos, compras, pagos, metodos-pago, bancos, billeteras, cuotas, planes-cuotas, promociones, promociones-bancarias.

Mensajería e Interacción: chats, mensajes, encuentros.

Moderación y Auditoría: reportes, actividades-sospechosas, bloqueos, moderacion-multimedia, estadisticas-publicacion.